EECS 581: Peer Enhancement

Project System Architecture

& Person-Hour Estimates

Version 1.2

Table of Contents

System Architecture Overview

Components

There are 6 major components to the architecture of the Minesweeper game: Board Manager, User Interface, Input Handler, Game Logic, AI mode, and Hints and Timer feature.

The Board Manager is responsible for creating/managing a 10x10 game board grid made through a 2D array. This component helps manage the game as a whole, as it aids with tracking cell status by indicating whether the cells are covered, uncovered, contain a mine/mine, whether it is flagged or unflagged, if there is a number, or if it is an empty cell. The board manager also aids with housing a User Interface component of the game.

The User Interface component is responsible for interacting with the user's actions and performing the correct tasks associated with each action. It is also responsible for showcasing the correct game status to the user, such as showcasing win and loss statuses. It aids with rendering the grid and making sure status indicators are visible to the user throughout the game (such as flag count, the number revealed on the grid, and clicking functionality). As a result, the User Interface component houses the Input Handles and Game Logic components.

The Input Handler component is responsible for processing the user input by identifying the specific actions that the user performs, such as left mouse-click indicates the user wants to select a cell to be uncovered, and right mouse-click indicates the user wants to place a flag on a cell. This component is responsible for communicating with the Game Logic component to carry out the user's desired actions.

Once the actions from the Input Handler component are communicated, the Game Logic component carries out various actions and handles gameplay rules depending on the different scenarios:

- The Game Logic will randomly generate mines only after the user's first click, and also make sure no mine is generated in the first clicked cell. This is to ensure that the user's first click is a guaranteed safe click.

- Based on where the mines are generated, the Game Logic will compute the correct number that will be placed in each mine-free cell to indicate to the user where a potential mine resides on the game board
- Game Logic will also attempt to recursively reveal adjacent cells (containing numbers) that are mine-free during the rest of the gameplay.
- Lastly, if all the mine-free cells are uncovered, the Game Logic detects a win, but if the user clicks on a mine, then all of the mines are revealed to the user and a loss status is detected.

The AI component is responsible for providing an interactive game-play environment by taking turns with the user. It consists of three subparts: easy, medium, and hard. When the user selects the Easy AI mode, the AI is responsible for randomly selecting a hidden cell to uncover. The Medium AI mode is responsible for applying 3 rules: 1. The AI should flag all the hidden neighbors if the number of hidden neighbors of a revealed cell equals the cell's number, 2. The AI should open all other hidden neighbors if the number of flagged neighbors of a revealed cell equals that cell's number, 3. If rules 1 and 2 are not applied, then the AI will choose a hidden cell randomly. Lastly, Hard AI is responsible for applying all of the rules that are a part of Medium AI; however, it should include the 1-2-1 rule. The 1-2-1 rules state that if three revealed cells are next to each other, and they are 1-2-1 (vertically or horizontally), then the AI should logically analyze the board to determine that the two outer covered neighbouring cells contain mines. As a result, these cells would be flagged. The inner cell that is hidden will still be open, as the AI should have determined that it is a safe cell. If none of these rules apply, then the AI should default to the Easy AI mode with the randomness feature.

The Hints and Timer feature of the game is responsible for enhancing the gaming experience for the user. The Hints feature will provide users with 2 hints for a safe cell, i.e., a cell that is covered and has a value of 0 hidden under it. Additionally, the timer provides the user with additional metrics of how long the user spent on each round.

Data Flow

For the game, the flow of data moves constantly between the user, the user interface (ui.py), the game board (board.py), and, as per the user's preference, the AI move selector as well. The main process of the game is that the user provides input, the interface interprets it, and the board processes it according to the rules of the Minesweeper game defined before sending the updated information back for display. This cycle repeats until the game ends in either victory or loss. The UML diagram we created (Diagram 1) illustrates this process, showing each component passes information to the next component to create a smooth playing experience. The flow of the program can be broken down into two parts: initial game flow and subsequent game flow.

At the beginning of the game, the user interacts with the user interface through the command line by pressing "Enter" to start. The interface asks for the number of mines that should be placed on the board, and once a valid number is given, it starts a timer to track the length of the game and calls the board component to create the grid. The board sets up a 10 x 10 arrangement of cells, but does not place any mines until after the first move, which ensures that the player's opening move is always safe. After placing the mines, the board calculates the numbers for each non-mine cell, where the number indicates how many mines are in the neighboring cells. The updated board is returned to the user interface, which prints it out to the screen so the player can see the starting state of the game along with the timer and the mine count.

In the subsequent game flow, the user continues to interact with the user interface. This is when the game enters its main loop, where the user continues to interact with the interface by entering commands in the terminal. These inputs might be typing coordinates along with a command to "reveal" a cell or to "flag" a potential mine, or giving one-word commands like "quit" to stop the game entirely, or "hint" to request a safe cell. The user interface receives and checks if the inputs are valid, translates them into board coordinates, and forwards the command to the board. The board then executes the command to reveal a cell, uncovers it, and if the cell is empty (indicated by the brown circle symbol), the board recursively uncovers neighboring cells until the numbered cells are reached. If the command is to flag a cell, the board changes the flagged state and updates the remaining flag count. If the user requests a hint, a special feature added by our team, the interface

signals the board to locate a guaranteed safe cell. The board then determines and returns if a safe move is possible, which the user interface prints for the player to see. To ensure the user doesn't cheat to win the game, there is a limit to how many hints the user gets in one full round, which is a maximum of 2 hints. At each step, the board's updated state is returned to the interface, which redraws the board and updates the statistics like the timer, mine count, and flags remaining.

Another feature we introduced is the AI system, which can be enabled at the beginning of the game. Once the AI mode is turned on, the user and the AI will take turns working to reveal cells on the board until the game is either won or lost. The AI chooses a cell to reveal or flag, depending on the selected difficulty level. In Easy AI mode, the AI selects a randomly covered and unflagged cell. In Medium AI mode, it applies two rules: if all hidden neighbors of a revealed number's cell must be mines, it flags them, and if all mines around a numbered cell are already flagged, it reveals the other hidden neighbors. If neither rule applies, it reverts to easy mode by choosing a random cell. Therefore, in medium mode, multiple moves are made by the AI in one turn, adding more difficulty for the user. In Hard AI mode, the AI extends these rules with more advanced reasoning between multiple cells by also applying the 1-2-1 rule after applying all rules from the medium AI mode. The 1-2-1 rule is when there are 3 consecutive cells, either vertically or horizontally, that have the numbers 1, 2, and 1 attached to them, respectively. This means that the two bombs are located in two of the corner neighbors of this 3-cell block, and the middle cell is safe. Therefore, after calling the medium AI mode, this function will transfer logic back to the hard AI mode to search for any such 1-2-1 combinations in the grid. If so, it will flag the corner tiles and reveal the safe neighbor. After every user or AI moves, the board checks whether the game has ended. If a revealed cell contains a mine, the board uncovers all mines and sends this result to the user interface, which prints a loss message and stops the timer. If the AI made the mistake of revealing the bomb, it would also print as such. If all safe cells are revealed, the board declares a win, and the user interface prints a win message and ends the game. If neither case occurs, the user interface prints the updated grid, and the game continues.

Overall, the flow of the game follows a defined and repeated path. The user provides input, the interface interprets it and forwards it, the board applies the game's rules and logic, and the results return to the interface to be displayed for the user.

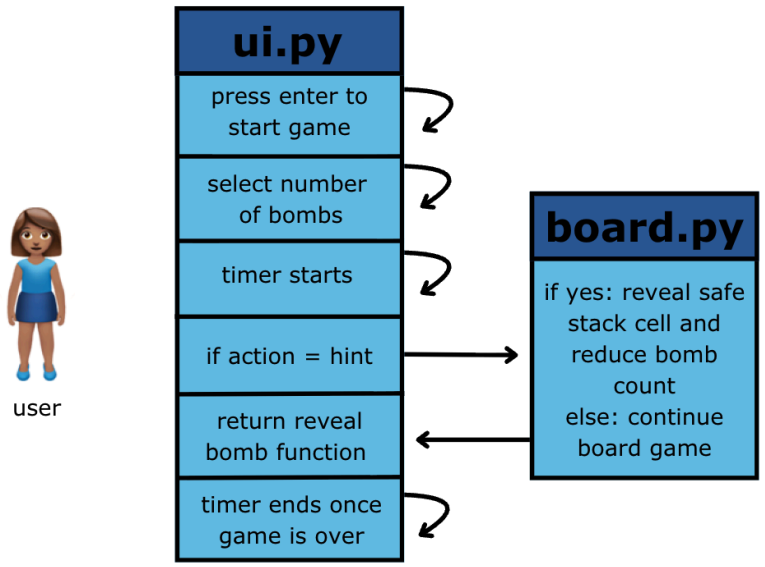

Diagram 1: Components and Data Flow

Key Data Structures

The core data structure for this Minesweeper project is the Board class. This class manages all aspects of the game, including the self.mines, self.state, self.adj, self.mine_total, self.playing_state, which are crucial to the board's functionality. The first three attributes are 2D arrays, where an entry in the array constitutes one of the rows in the grid, and each element in one of these rows represents one of the cells. Therefore, each array has a dimension of 10 by 10, totaling 100 elements, where each cell is indexed by row (i) and column (j). The self.mines attribute stores whether the cell has a mine, indicated by True or False. All cells are initialized as False and updated after the first move when the mines are placed. The second attribute, self.state, tracks the state of the particular cell, which may be “COVERED”, “FLAGGED”, or “UNCOVERED”. Initially, all cells are set to “COVERED”, and this attribute updates as the player or AI interacts with the board, allowing safe cells to be uncovered recursively. The third array, self.adj, records the number of adjacent mines, providing hints to the user. The value ranges from -1 to 8, where -1 indicates that a mine exists in

that cell. These values are initialized to 0 and are calculated and updated once by the compute_numbers() function after the mines are placed.

In addition to the arrays, the class includes two other important attributes. self.mine_total stores the number of mines chosen at game initialization, which can range from 10 to 20. self.playing_state keeps track of whether the game is ongoing, won, or lost, allowing the UI class to update the terminal with the appropriate display.

Together, these key attributes of the Board data structure ensure the board functions correctly and the game state is consistently maintained.

Aiding Future Extensions

To aid future extensions, our team added robustness to our code to ensure that there would be ease in logic and error handling. For example, when we added the 3 different AI features to the Minesweeper game, we wanted to ensure that the internal game logic kept track of which rules in each difficulty level were being used. For example, there are two different strategic rules that get applied to the Medium AI level. Rule 1 is where the AI checks whether the number of hidden neighbors of a cell equals the cell's number, and then flags them. Rule 2 is where the AI checks if the number of flagged neighbors equals the cell's number; then it should reveal all the other covered neighbors. These two rules may not apply to each of the moves that are made throughout the game. Therefore, it is important to keep track of the different rules that are used to determine if random mode needs to be used or not. For example, in Hard AI mode, it will use the Medium AI's functionality and apply its own 1-2-1 rule. If no rule applies at all from medium mode or hard mode, then it must default to finding a random cell to reveal. For future extensions, we also returned what rule that has been applied when medium mode is called in hard mode. This can help in later endeavours to add more features to the code, if the team wants to add different rule sets to the AI based on what rules have already been applied.

Another assumption that can be made about the AI solver functionality includes when the AI solver gets invoked. For example, if the user only wants to play by themselves, then there is a mode that they can choose to not have AI logic in the game. Otherwise, they can choose to play with AI and

can decide what mode they want. This can help with future iterations if the team would like to add another feature to have just the AI solve the board without any user logic, for example. This would be a relatively easy addition, as we already have a user input variable in ui.py to ask if the user wants to play with AI. This can be extended to ask if the user only wants AI to play, and add a third option in the data logic.

Finally, our team also prints out each move that the AI makes every time it takes a turn so that the game logic is more intuitive for the user when they play, rather than just printing out the final board once the AI makes its move. This can help with strategizing during the game, ensuring that the rules are applied properly, and any error checking and testing in the future.

Overall, our team has intentionally added robustness, commented our code, and refactored to ensure that there was a strong base to build new features pertaining to the Minesweeper game in the future. This included error checking and detailed user input logic.

Person-Hour Estimates

Methodology

First, we went through the project's code to understand how the previous group implemented their Minesweeper game. Then we ensured they had the correct functionality from Project 1 requirements. We then ranked the additional required features based on the complexity of each component and the dependencies on other parts of the code.

From there, we were able to create a backlog of ticket issues that contained all the project requirements that were needed to enhance the game. Then, using the methodology learned in class, called "Planning Poker", we did a preliminary round of estimation of person-hours. We accounted for buffer time, meetings for planning sprints, and testing the code.

We estimated each requirement by having each person vote on the number of hours they thought it would take to complete the ticket. If there were disputes on the ticket estimation from different people, each person would explain why they thought it would take that many person-hours, and we

would do another round of estimation. Once we had reached a consensus about the ticket, that would be our final estimate.

Tickets	Estimated Person-Hours
Easy AI Solver Level #50	3 hours
Medium AI Solver Level #51	3 hours
Hard AI Solver Level #52	3 hours
User Input of Difficulty Level #53	3 hours
Additional Feature - Hint Feature #54	3 hours
Create a Timer for Minesweeper #58	1 hour
Print the Board Key #57	1 hour
Commenting and Cleaning Code #56	1 hour
System Architecture + UML Diagram #55	1 hour

Actual Person-Hours

Tickets	Actual Person-Hours	Date	Assignee
Easy AI Solver Level	1 hour	09/23/2025	Anna
Medium AI Solver Level	5 hours	09/26/2025 09/27/2025	Nikka
Hard AI Solver Level	6 hours	09/23/2025 09/27/2025	Kusuma

User Input of Difficulty Level	6 hours	09/23/2025	Nimra
Additional Feature - Hint Feature	3 hours	09/29/2025	Sophia
Create a Timer for Minesweepers	1 hour	09/25/2025	Sophia
Print the Board Key	<1 hour	09/23/2025	Nikka
Commenting and Cleaning Code	1 hour	09/29/2025	All
System Architecture + UML Diagram	1 hour	10/01/2024	All

Additional Actual Person-Hours from Group Meetings:

09/23/2025: 1.5 hours
09/27/2025: 4.5 hours
10/01/2025: 1 hour

All members of the Group 30 team will regularly manage, update, and commit to the GitHub Repository. The repository will be publicly available for viewing and accessing here: P2 Enhancement Minesweeper Group 30 GitHub.

**** Note: All our Project Meeting Logs will be housed in the GitHub Repository on the Wiki Page. Please reference it as needed. We also have our Sprint Meetings and tickets created on the GitHub page. Note that the branch used for Project 2 is called project2_main.**